Groups

You can tell me the composition of the groups in the forum

- 3 to 5 students
- no hurry, but if you have it already just drop the list and name of the group there
- and if you don't have a group, you can start to look around
  - use the forum, ask classmates, etc etc..
  - avoid to pop up at the end saying "I don't have a group, I'll do it alone"
    - Much more work
    - Previous experience shows lower grades
    - And software architectures are a matter of team play…

Ca' Foscari University of Venice

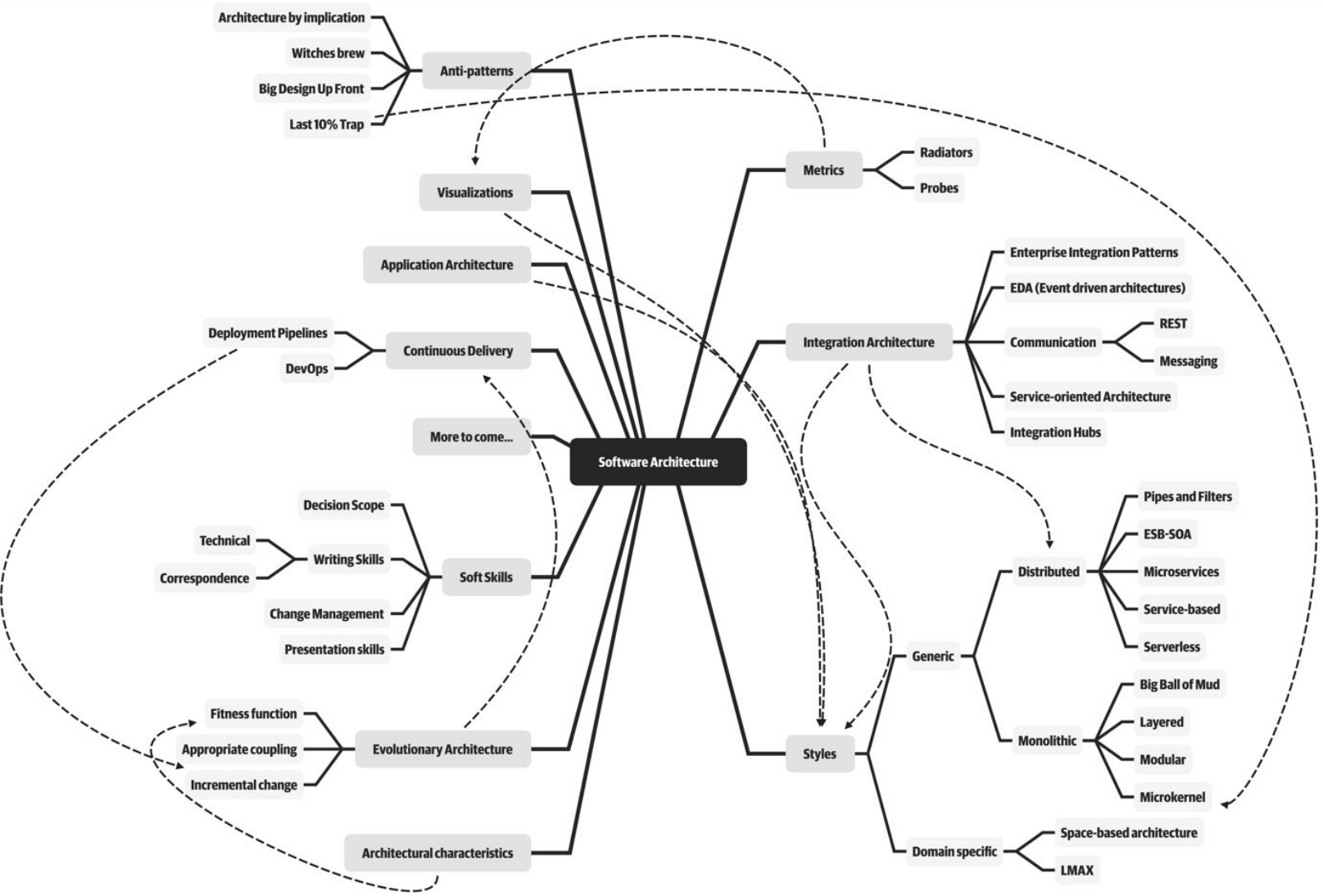
An introduction to software architect[ure] [Concepts]

Software architectures

Pietro Ferrara
pietro.ferrara@unive.it

Ca' Foscari University of Venice

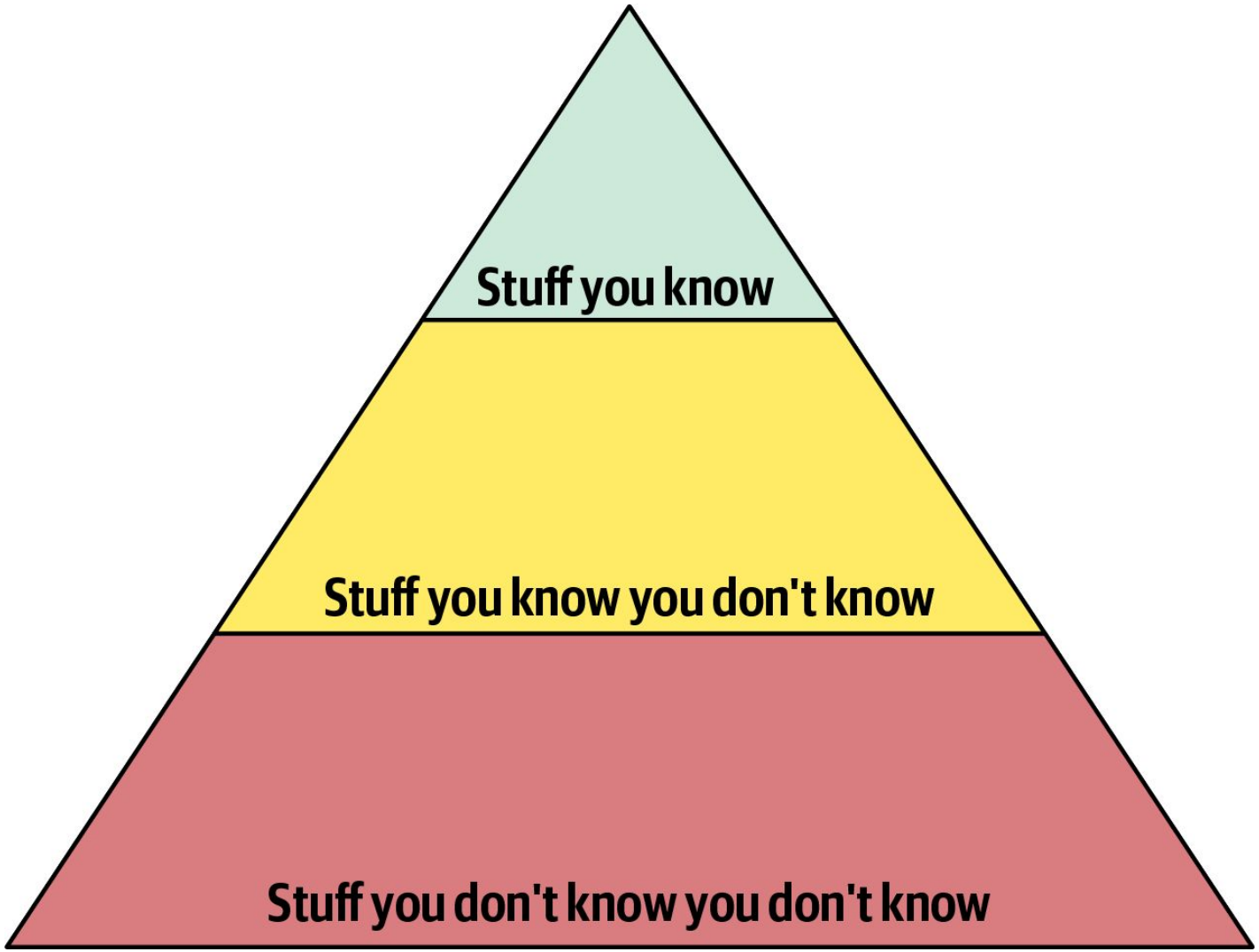
Skills



- Technical low level knowledge
- Technical high level knowledge
- System knowledge
- Soft skills

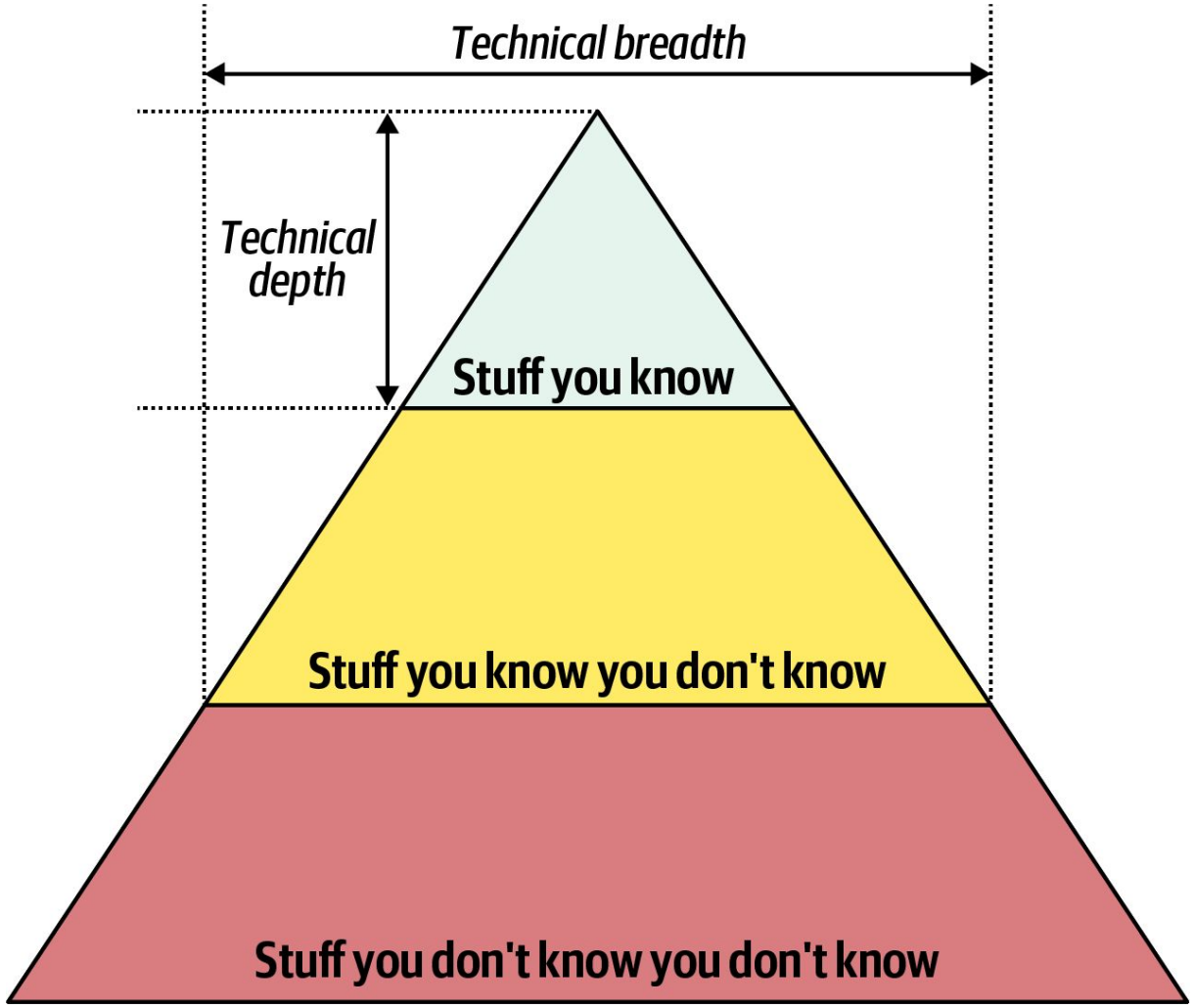
- Nobody can have all those skills together!!!

Technical knowledge



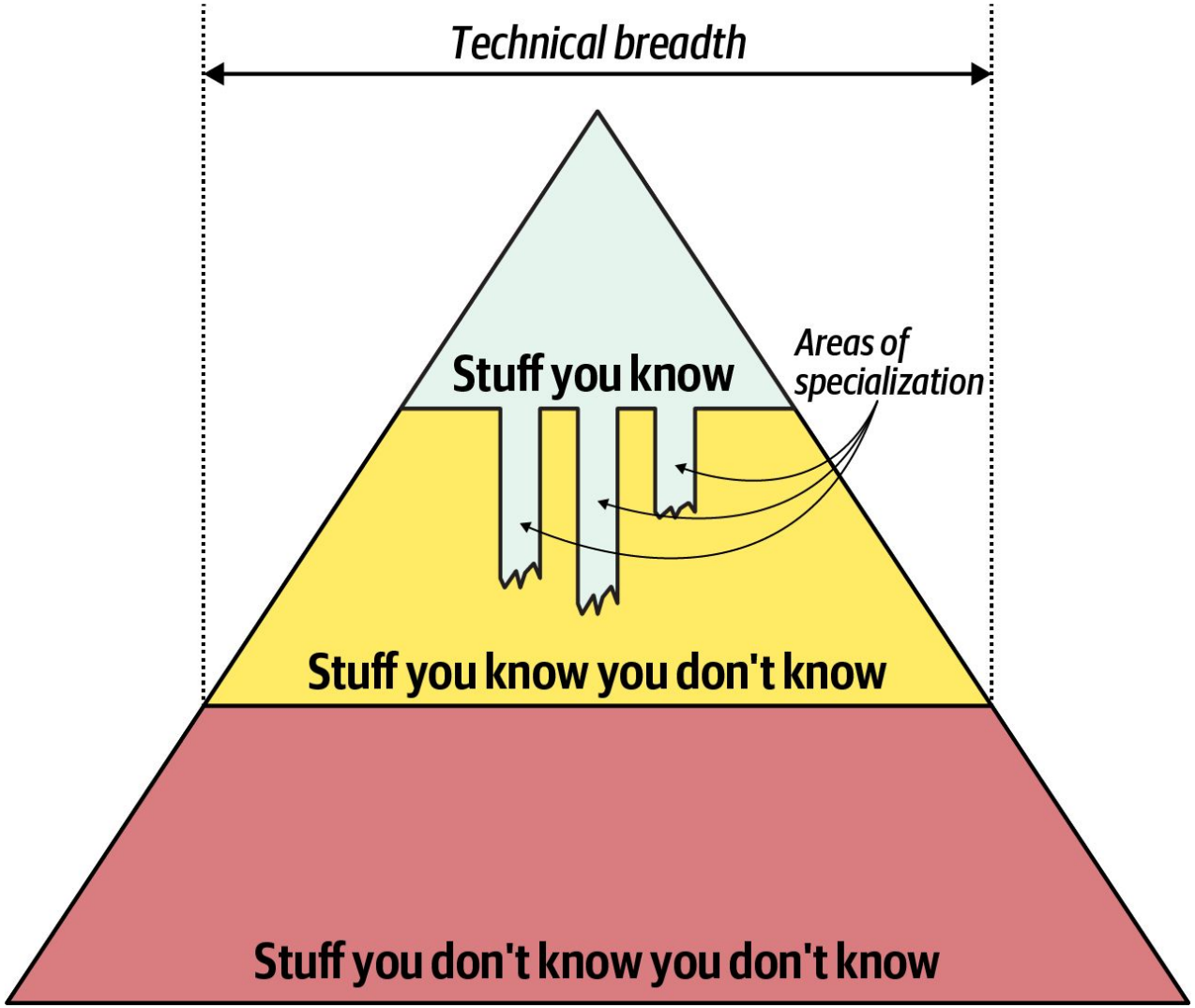
- Technical knowledge is extremely wide
- Much more stuff you don't know than you know
- Even worse: you ignore even the existence of the most part of technologies
- Should you be ashamed?
  - No!!!
  - We are all on the same boat

Technical debt vs breadth



- Developers: depth!
  - What you know
- Architects: breadth!
  - How much you know
- Uncomfortable: the more you learn, the more you don't know
- Depth to implement
- Breadth to take decisions
  - And then delegate implementation
- Breadth is more important than depth for an architect

Areas of specialization



- Don't completely give up
  - Important to keep some specific areas of expertise
- But be careful about the rest
  - Your knowledge will become outdated (very) soon
  - Don't make decisions with such an outdated knowledge
- Architecture evolves very fast
  - I found only books published after 2020 interesting for this course

Ca' Foscari University of Venice

Software developers career

CTOs

Software architects

Project managers

Software engineers

Software developers

Ca' Foscari University of Venice

From sw developer to sw architect

- A rather standard career promotion
  - Most of software architects were developers
- However, sw architects need to have different skills
  - Developers spend their time coding
    - Technical depth
  - Architects understands business needs, take architectural decisions, manage a team
    - Technical breadth + soft skills
- Good developer does not imply good architect
- Hard to find your path to architect…
- Some good resources: https://developertoarchitect.com/

Ca' Foscari University of Venice

Soft skills

Soft skills, also known as common skills or core skills, are skills applicable to all professions. These include critical thinking, problem solving, public speaking, professional writing, teamwork, digital literacy, leadership, professional attitude, work ethic, career management and intercultural fluency. This is in contrast to hard skills, which are specific to individual professions.
https://en.wikipedia.org/wiki/Soft_skills

WIKIPEDIA
The Free Encyclopedia

- Soft skills are needed to be a good software architect
  - Need to manage a team, interface with customers, etc etc..
- But this course is about software architecture, not soft skills!
- But from time to time we'll discuss things related to soft skills

Requirements of IT systems

- A requirement of an IT system is either
  - A service that must be provided
    - E.g., given two numbers a and b, returns a+b
  - A characteristic that must be satisfied
    - E.g., being able to perform 1 million of additions per second
- Functional vs non-functional requirements
  - Architecture is about taking decisions to fulfill (some) non-functional requirements
  - Functional requirements are completely left out from this course
- Topics already covered in standard sw engineering courses
  - https://moodle.unive.it/course/view.php?id=7859

Architecture characteristics

- Textbook: we dislike that term (nonfunctional requirements) because it is self-denigrating. We prefer architecture characteristics
- I personally don't care!
- Nonfunctional requirements are widely accepted
- Architecture characteristics look fine as well
- Let's stick to the textbook's choice
  - But keep in mind that usually they are called nonfunctional requirements
- Auditability, performance, security, requirements, scalability, …

Criteria

Architecture characteristics **must** meet three main criteria:

1. Specifies a nondomain design consideration
   - Operational and design criteria for success
   - How to implement the requirements and why some choices were made
2. Influences some structural aspect of the design
   - Requires special structural consideration to succeed
3. Is critical or important to application success
   - Each characteristic adds complexity
   - Focus only on the ones that are essential

A non-exhaustive questionable list follows!

Operational architecture characteristics

Term	Definition
Availability	How much of the time the system will need to be available; if that's 24/7, steps need to be in place to allow the system to be up and running quickly in case of any failure.
Continuity	The system's disaster recovery capability.
Performance	How well the system performs; ways to measure this include stress testing, peak analysis, analysis of the frequency of functions used, and response times.
Recoverability	Business continuity requirements: in case of a disaster, how quickly the system must get back online. This includes backup strategies and requirements for duplicate hardware.
Reliability/ safety	Whether the system needs to be fail-safe, or if it is mission critical in a way that affects lives. If it fails, will it cost the company large sums of money? This is often a spectrum rather than a binary.
Robustness	The system's ability to handle error and boundary conditions while running, for example, if the internet connection or power fails.
Scalability	The system's ability to perform and operate as the number of users or requests increases.

Structural architecture characteristics

Term	Definition
Configurability	How easily end users can change aspects of the software's configuration through interfaces.
Extensibility	How well the architecture accommodates changes that extend its existing functionality.
Installability	How easy it is to install the system on all necessary platforms.
Leverageability/reuse	The extent to which the system's common components can be leveraged across multiple products.
Localization	Support for multiple languages on entry/query screens in data fields.
Maintainability	How easy it is to apply changes and enhance the system.
Portability	The system's ability to run on more than one platform (such as Oracle and SAP DB).
Upgradeability	How easy and quick it is to upgrade to a newer version on servers and clients.

Cloud architecture characteristics

Term	Definition
On-demand scalability	The cloud provider's ability to scale up resources dynamically based on demand.
On-demand elasticity	The cloud provider's flexibility as resource demands spike; similar to scalability.
Zone-based availability	The cloud provider's ability to separate resources by computing zones to make for more resilient systems.
Region-based privacy and security	The cloud provider's legal ability to store data from various countries and regions. Many countries have laws governing where their citizens' data may reside (and often restricting it from storage outside their region).

Cross-cutting architecture characteristics

Term	Definition
Accessibility	How easily all users can access the system, including those with disabilities like colorblindness or hearing loss.
Archivability	The system's constraints around archiving or deleting data after a specified period of time.
Authentication	Security requirements to ensure users are who they say they are.
Authorization	Security requirements to ensure users can access only certain functions within the application (by use case, subsystem, web page, business rule, field level, etc.).
Legal	The legislative constraints in which the system operates, such as data protection laws like GDPR or financial-records laws like Sarbanes-Oxley in the US, or any regulations regarding the way the application is to be built or deployed. This includes what reservation rights the company requires.
Privacy	The system's ability to encrypt and hide transactions from internal company employees, even DBAs and network architects.
Security	Rules and constraints about encryption in the database or for network communication between internal systems; authentication for remote user access, and other security measures.
Supportability	The level of technical support the application needs; to what extent logging and other facilities are required to debug errors in the system.
Usability/ achievability	The level of training required for users to achieve their goals with the application/solution.

Ca' Foscari University of Venice

ISO 25010 list of capabilities

https://iso25000.com/index.php/en/iso-25000-standards/iso-25010

Slightly different and overlapping proposals. We will focus on a given (sub)set.

Ca' Foscari University of Venice

Our architecture characteristics

Overall cost	Additional effort required to implement the system
Partitioning type	Technical or domain driven
Simplicity	The ease of understanding the overall system
Modularity	Degree to which the software is composed of discrete components
Maintainability	The ease of modifying the software to improve, correct or adapt it
Testability	The ease of and the completeness of testing

Deployability	The ease, frequency, and risk of deployment
Evolvability	The ease of evolve the software
Responsiveness	How much quick is the software to reply
Scalability	Ability for the system to perform and operate as the number of users or requests increases
Elasticity	The ability to handle bursts of users
Fault tolerance	Does the software operate as intended despite hardware or software faults

Structural architecture characteristics **Engineering architecture characteristics** **Operational architecture characteristics**

Which characteristics?

Never shoot for the *best* architecture, but rather the *least worst* architecture.

- We'd like an IT system that is scalable, reliable, efficient, modular, cheap to implement, etc..
  - But we should focus on what it really matters!
  - (and it is not feasible anyway)
- Minimal set of architecture characteristics
  - With a minimal level of objectives
  - That are needed for the success of the IT system

So far so good… maybe!

- The list of architecture characteristics is given
  - There does not exist a clear description of each one
    - Even if it's the list chosen in the textbook!
  - The boundaries are unclear
    - Reliability vs fault tolerance? Elasticity vs scalability?
  - The description is anyway informal
    - Thus ambiguous
- We need to establish some metrics to evaluate them
  - But this is not possible on a generic description of the architecture
  - Think about that during the course to make all this more concrete!

Ca' Foscari University of Venice

SMART objectives

- When we set up an object we should have that it is:
  - Specific: target a specific area for improvement.
  - Measurable: quantify or at least suggest an indicator of progress.
  - Assignable: specify who will do it.
  - Realistic: state what results can realistically be achieved, given available resources.
  - Time-related: specify when the result(s) can be achieved.
- https://en.wikipedia.org/wiki/SMART_criteria
- We are still missing measurable
  - And we are going to underspecify it, but we must discuss…

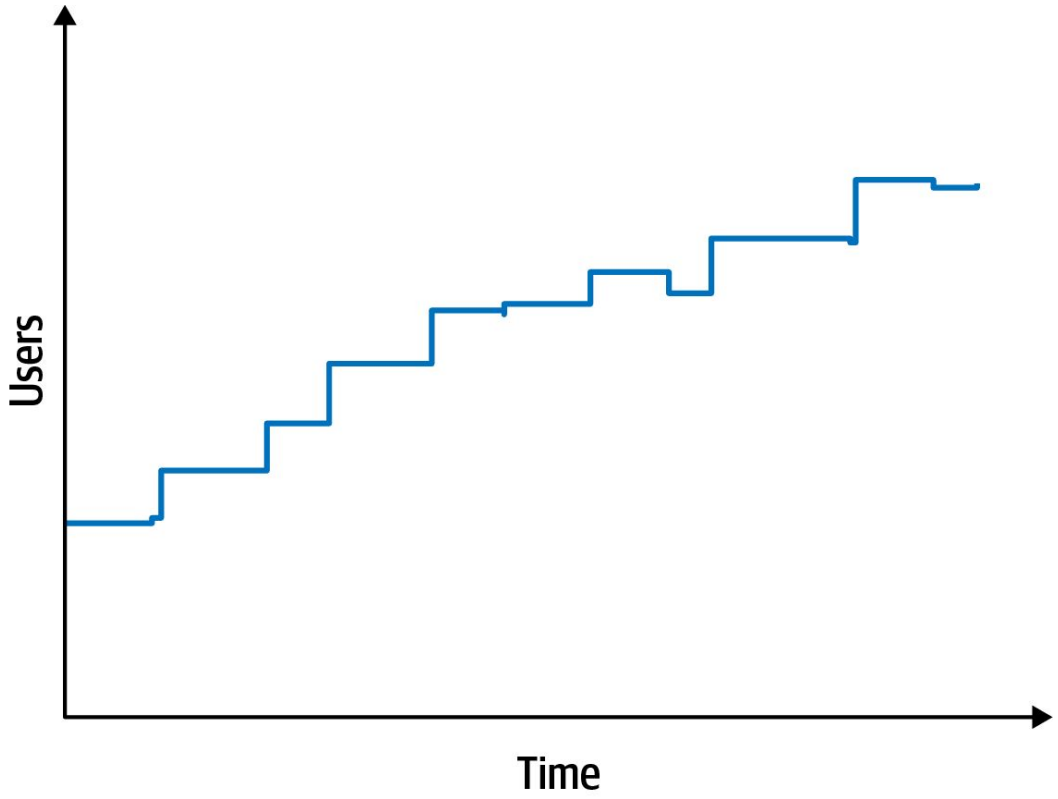
Measuring architecture characteristics

- Architecture characteristics are not physics
  - There is no absolute way to measure them
  - Different people will interpret them differently
  - The target might be different in different systems
    - E.g., scalable to 1K or 1M or 1B users?
- Ideally, automatically assess their achievement
  - Integrate this in the DevOps lifecycle as tests
- For each characteristic we choose, we must define a measure
  - In an agile way: set a number, iterate the DevOps lifecycle, and then modify it based on the feedback

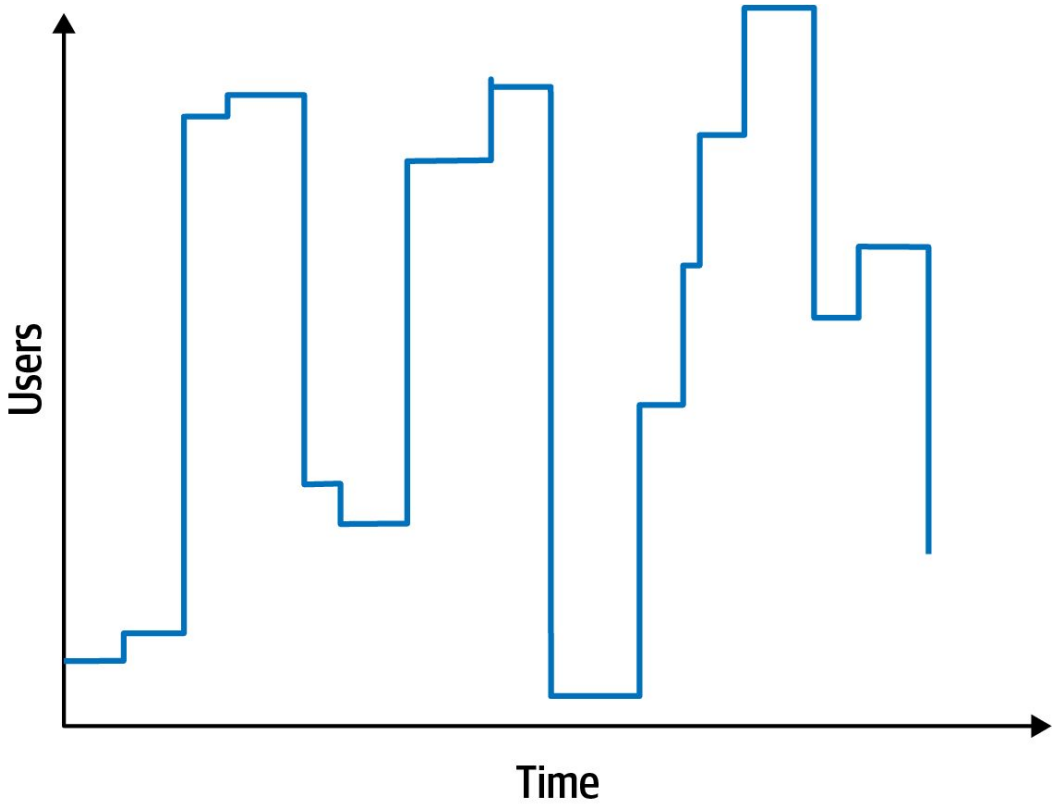
Operational characteristics

***Elasticity**: the ability to handle bursts of users*
***Performance**: amount of used resources, including time behavior, resource utilization,...*
***Reliability**: degree to which a system functions under specified conditions for a specified period of time*
***Scalability**: ability for the system to perform and operate as the number of users or requests increases*

Scalability vs Elasticity



Scalability



Elasticity

Your wooclap poll will be displayed here

1

Install the **Chrome** or **Firefox extension**

2

Make sure you are in **presentation mode**

wooclap

https://app.wooclap.com/events/SADM/0 questions 8-11

Operational characteristics

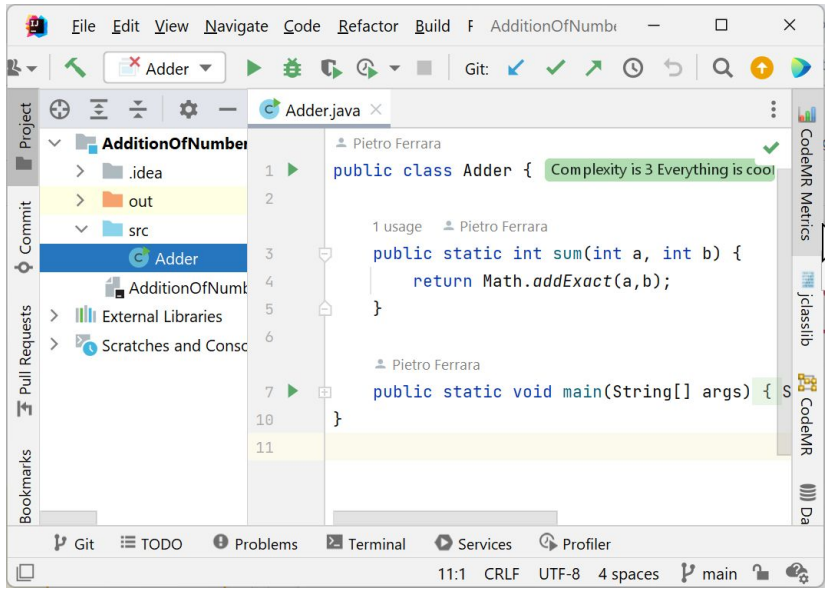
***Elasticity**: the ability to handle bursts of users*
***Performance**: amount of used resources, including time behavior, resource utilization,...*
***Reliability**: degree to which a system functions under specified conditions for a specified period of time*
***Scalability**: ability for the system to perform and operate as the number of users or requests increases*

Elasticity: how many more/less users the systems receive in a time frame
Performance: how much resources (in terms of time, memory, processor, hard drive, network, …) are consumed per request/user/task/…
Reliability: how many tasks I can perform under specific stress conditions (network slowdown, massive amount of users/requests, …)
Scalability: how many more users/requests/tasks the system is capable to handle over units of time

Ca' Foscari University of Venice

Technologies

Source code

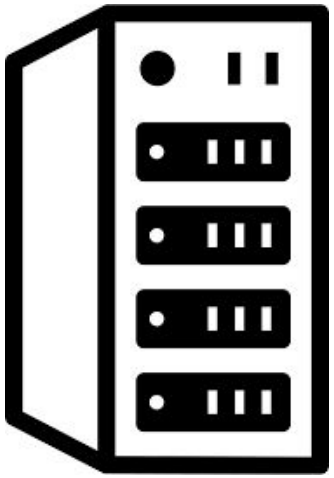


Artifact

Compile and build

Deploy

Server



Ca' Foscari University of Venice

Katas

https://nealford.com/katas/list.html

- From where do we start when thinking about an IT system?
- Let's adopt the approach of katas
  - In martial arts, it is an individual exercise
- High-level domain targeted description of an IT system
- Next step: derive architecture characteristics from katas
- Katas are composed by:
  - Description: The overall domain problem the system is trying to solve
  - Users: The expected number and/or types of users of the system
  - Requirements: Domain/domain-level requirements, as an architect might expect from domain users/domain experts
  - Additional context

First task

Implement an IT system that manages sport tournaments, e.g.,

- enrollment, pairing, results, standings, reports of single matches, website of the event, …
- not bound to specific sports, up to you to restrict it to
  - a specific sport (e.g., soccer, volleyball, chess, …)
  - a category of sports (e.g., individual or team)
  - a "bundle" of sports (e.g., olympic games, universiadi, …)
- Focus on a subset of (functional and nonfunctional) features
  - You will need to implement the system at the end!

Structural characteristics

***Deployability**: the ease, frequency, and risk of deployment*
***Fault tolerance**: does the software operate as intended despite hardware or software faults*
***Simplicity**: the ease of understanding the overall system*
***Testability**: the ease of and completeness of testing*

- Deployability: automatically deployed + time required
- Fault tolerance: simulation of faults
- Simplicity: hard to measure… (see next slide)
- Testability: automatic testing + time required + code coverage

Measuring modularity

***Modularity**: degree to which the software is composed of discrete components*

- Module: each of a set of independent units that can be composed to construct a more complex structure
  - Group of classes in object-oriented programming languages
    - E.g., packages or modules in Java, namespaces in C#
  - Group of functions in imperative or functional programming languages
- Logical distinction, not necessarily a physical one
  - Even if modules usually are shipped through artifacts (e.g., jar files)
- Modules might depend on each other
  - But in general we cannot have static cyclic dependendencies

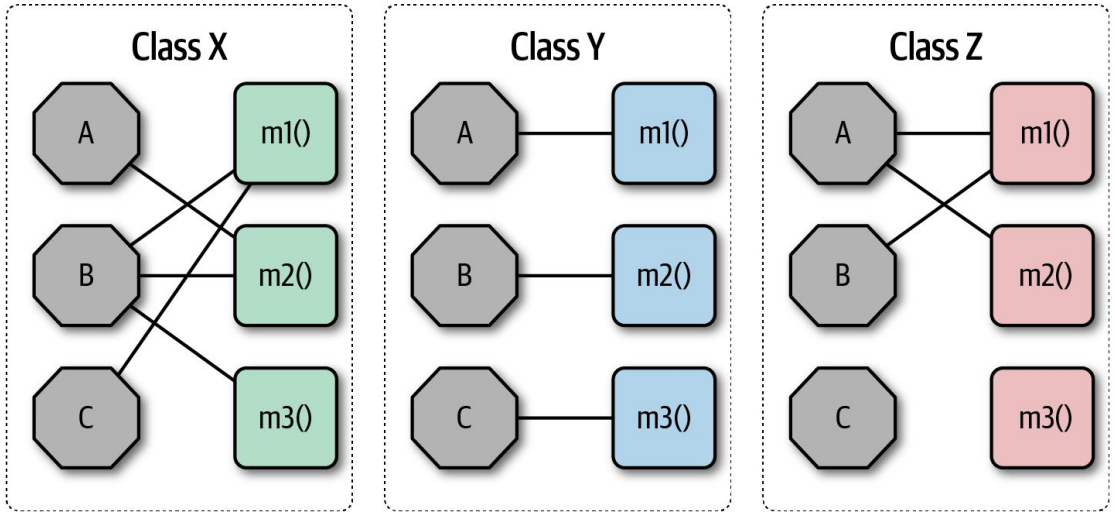
Measuring modularity
Cohesion

***Modularity**: degree to which the software is composed of discrete components*

- Cohesion measures how related the parts are to one another
- Many different types: functional, sequential, communicational, procedural, temporal, etc…
- Chidamber and Kemerer Object-oriented metrics suite
  - https://en.wikipedia.org/wiki/Programming_complexity#Chidamber_and_Kemerer_Metrics
  - Lack Of Cohesion Metric

$$LCOM = \begin{cases} |P| - |Q|, & \text{if } |P| > |Q| \\ 0, & \text{otherwise} \end{cases}$$

- P: methods not accessing a shared field
- Q: methods sharing a shared field



High Low Medium

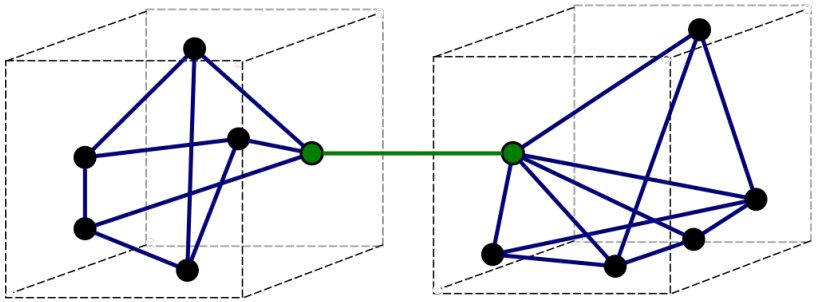
Ca' Foscari University of Venice

Measuring modularity Coupling

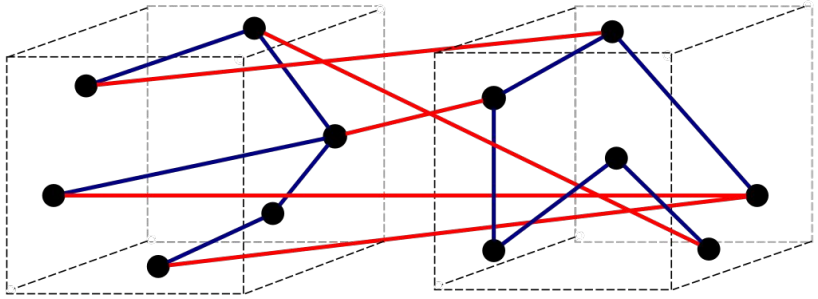
***Modularity**: degree to which the software is composed of discrete components*

Attempting to divide a cohesive module would only result in increased coupling and decreased readability. (Larry Constantine)

- Coupling: the degree of interdependence between modules
- Afferent: the number of incoming connections to a code artifact
- Efferent: the number of outgoing connections to other code artifacts



a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

Measuring modularity Coupling

***Modularity**: degree to which the software is composed of discrete components*

- Abstractness: measures the amount of abstract elements
  - Too low: just a main method
  - Too high: hard to reason

$$A = \frac{\sum m^a}{\sum m^c}$$

#abstract elements (abstract classes, interfaces, …)

#concrete elements (non abstract classes)

- Instability: % efferent coupling
  - Too high: code breaks more easily when changed
    - Because it delegates most of the work to other modules

$$I = \frac{C^e}{C^e + C^a}$$

efferent (outgoing) coupling

afferent (incoming) coupling

Measuring modularity Coupling

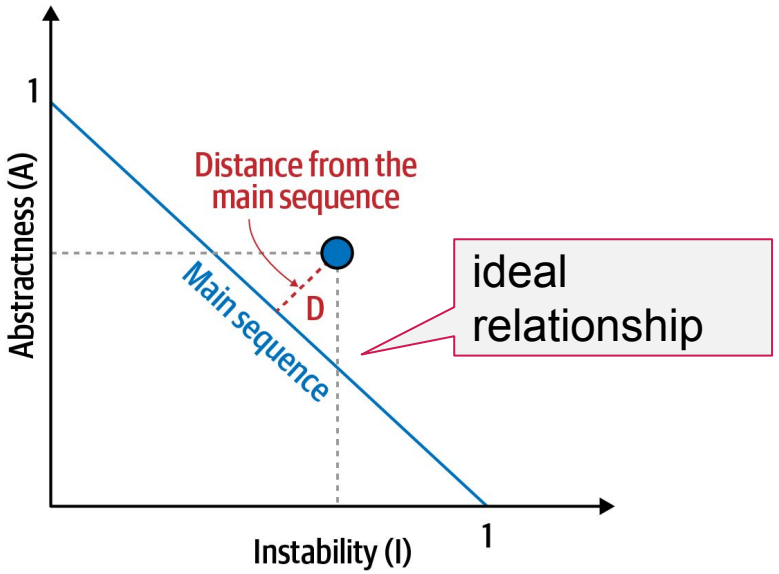
***Modularity**: degree to which the software is composed of discrete components*

- Abstractness and instability always fall between zero and one
  - In practical contexts… maybe…
- More abstract -> more stable
- If above the line, uselessness
  - Too abstract and too instable, impossible to reason on it
- If below the line, pain
  - Too much implementation and not enough abstraction, hard to mantain

abstractness

$$D = |A + I - 1|$$

instability



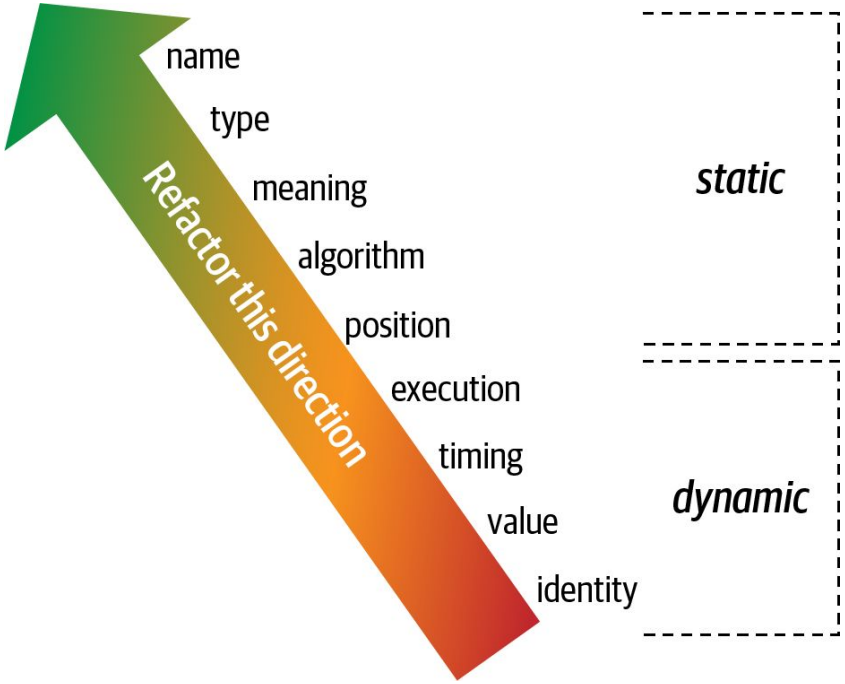
Ca' Foscari University of Venice

Measuring modularity Connascence

***Modularity**: degree to which the software is composed of discrete components*

Two components are connascent if a change in one would require the other to be modified in order to maintain the overall correctness of the system (Meilir Page-Jones)

- Static: source code level coupling
  - Need to modify the code
  - Name, type, position, algorithm, …
- Dynamic: execution/runtime coupling
  - Order of execution, timing, values, …



Ca' Foscari University of Venice

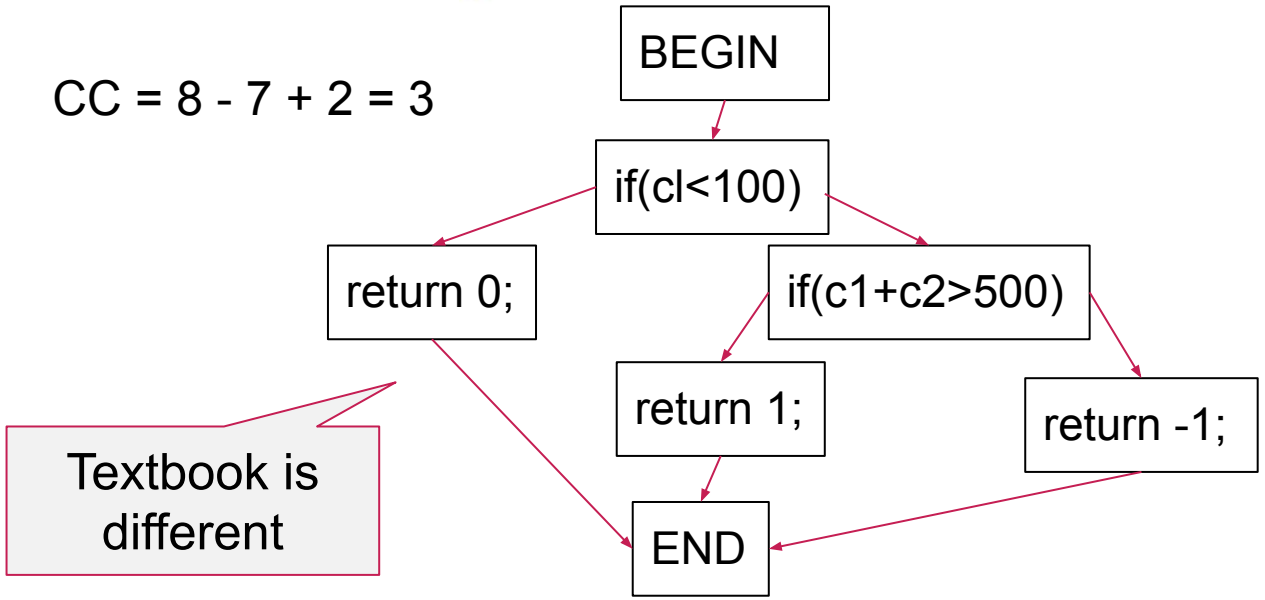
Measuring modularity
Cyclomatic complexity

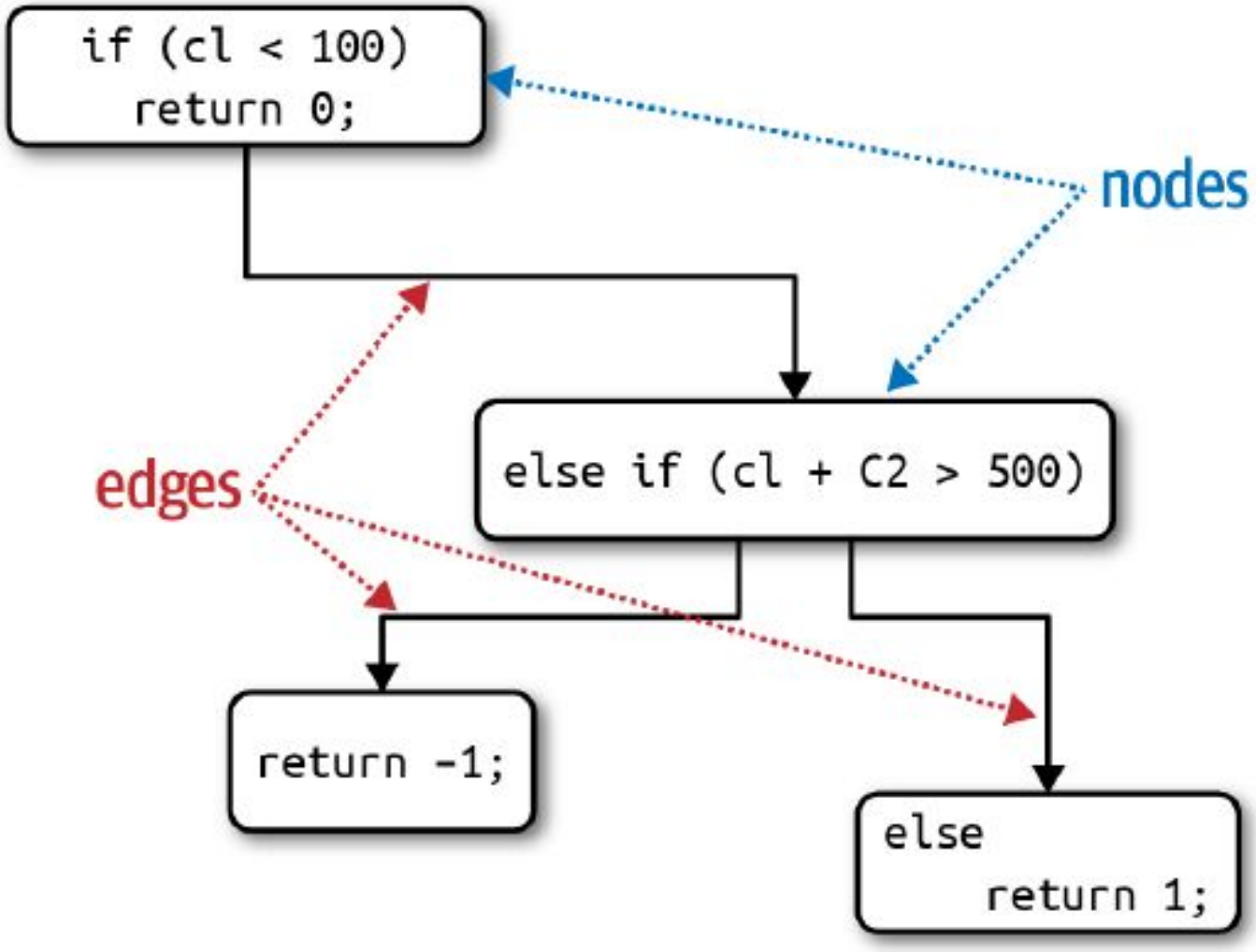
***Modularity**: degree to which the software is composed of discrete components*

- Measure the structure of the code
- Cyclomatic complexity
  - CC = E - N + 2
- Program as a (control flow) graph
  - N: number of lines of code
    - Statements or sequential blocks
  - E: number of "jumps"
    - E.g., if statements have two jumps
- Questionable anyway but
  - Provides a clear index
  - It can be automatically assessed

```
public void decision(int c1, int c2) {
    if (c1 < 100)
        return 0;
    else if (c1 + C2 > 500)
        return 1;
    else
        return -1;
}
```

CC = 8 - 7 + 2 = 3




if (cl < 100)
return 0;
nodes
else if (cl + C2 > 500)
edges
return -1;
else
return 1;

Ca' Foscari University of Venice

Summing up

Characteristic	What is needed			What it concerns		
	Code	Artifact	Deployment	Speed	Robustness	Structure
Deployability		X				X
Elasticity			X	X		
Evolutionary						X
Fault tolerance			X		X	
Modularity	X					X
Overall cost						
Performance		X		X		
Reliability		X	X		X	
Scalability			X	X		
Simplicity	X					X
Testability		X			X	

Ca' Foscari University of Venice

From domain concerns to architecture characteristics

- Need to find an agreement with domain stakeholders
  - But software architects talk a different language
- Architecture characteristics mapped to domain concerns?
  - It is not a one to one relationship
- Let's focus on few domain concerns:
  - mergers and acquisitions, time to market, user satisfaction, competitive advantage, time and budget
- What architecture characteristics we need?
  - Elasticity, Performance, Reliability, Scalability, Deployability, Fault tolerance, Simplicity, Testability, Modularity

Textbook "solution"

Domain concern	Architecture characteristics
Mergers and acquisitions	Interoperability, scalability, adaptability, extensibility
Time to market	Agility, testability, deployability
User satisfaction	Performance, availability, fault tolerance, testability, deployability, agility, security
Competitive advantage	Agility, testability, deployability, scalability, availability, fault tolerance
Time and budget	Simplicity, feasibility

Ca' Foscari University of Venice

Explicit characteristics from requirements

- Sometimes requirements make explicit the characteristic
  - E.g., "need to deal with ~10Ks concurrent users"
- Consider each single requirement in the kata/specification
  - Understand what consequences it has on characteristics
  - Both requirements and users sections are relevant
- However, carefully consider what is impacted
  - E.g., "the IT system is expected to get 100Ks users"
    - Concurrent users? Always or just in some timeslots (e.g., promotion)?
    - Elasticity vs scalability
- Always discuss with the stakeholders what they want

Implicit characteristics

- Unfortunately, stakeholders leave many things implicit
- E.g., probably nobody will put a requirement on security
  - But are you willing to get an unsecure IT system at the end?
- Another common example is availability
  - One expects that the system will be available <u>most of the time</u>
- But more characteristics lead to more complexity
  - That lead to higher costs!
- Common sense rules
  - Make explicit all the implicit requirements with the stakeholders
  - Choose the requirements with the highest priority

References

- Textbook, Part I
  - Technical depth/breadth: chapter 2
  - Architecture characteristics: chapter 4
  - Measuring architecture characteristics: chapter 6
  - Measuring modularity: chapter 3
  - From requirements to architecture characteristics: chapter 5